Exponential time hypothesis is false… but the verified algorithm is slower than trying all 2^n assignments at every practical size.

Alex Linhares
Argo Lab AGI; on leave from the Getulio Vargas Foundation
alex@argolab.org ORCID 0000-0002-4227-6879

November 5, 2026

Abstract

Placeholder. Item 09 writes the abstract (under 200 words; no number without a section that regenerates it). The title's sentence made precise: what is verified is a guarantee with existential constants and the exponent $n(\log\log n)^2/\sqrt{\log n}$, whose factor exceeds 1 for every $50 \le n < e^{5504}$, so the verified guarantee is weaker than the trivial $2^n \cdot \text{poly}$ guarantee at every practical size; the machine's running time is unknown because the constants are never computed; the manuscript's sharper exponents are unverified; the statement does refute ETH; the audit found no flaw.

1 Introduction

Placeholder. Item 09 drafts this section: the title's sentence made precise in the first paragraph (a guarantee, not a running time; existential constants never computed; $\rho(n) > n$ for every $50 \le n < e^{5504}$; the sharper exponents unverified); the one-paragraph summary; what is claimed and what is not; the stance (a careful reading by people who checked; no flaw found). Ends with what this does not claim".

2 The verified statement, read exactly

Placeholder. Item 02 drafts this section: the comparator's definitions one by one with their lines, the quantifier order, the encoding length, the exponent `runtimeScale`; the table (label `tab:runtime-scale`) and figure (label `fig:runtime-scale`) of $\rho(n)/n$ (from `paper3/scripts/runtime\_scale.py`); the title's claim stated precisely. Ends with what this does not claim".

3 What the manuscript claims beyond the verified statement

Placeholder. Item 03 drafts this section: Theorem 1.1's two sharper exponents, Corollary 1.4's quantitative circuit bound, Appendix C's consequences, Appendix D, each tagged verified or paper-only with page and line of the extract. Ends with what this does not claim".

4 The proof, inspected

Placeholder. Item 04 drafts this section: the closure size, the zero counts, the 33 identical definitions, the compile lemmas concluding in the comparator's own `Trace`, the unpatched Mathlib, the absence of a recorded axiom audit, of CI and of a second kernel; joined with the replay of Section 5. Ends with what this does not claim".

5 Independent replay

The repository holds no recorded `#print axioms` output for the family, no continuous integration that builds its Lean, and `enable_nanoda: false` in the comparator's JSON (Section 4). We therefore built the solution's import closure ourselves, in isolation, at the toolchain the repository pins, and recorded what the kernel reports. This section says exactly what was run, where, and what came out; the scripts are quoted verbatim in `paper3/replay/README.md`, and the logs are under `paper3/replay/logs/`.

5.1 What was run, and where

A fresh copy of the clone (commit `4eaee875`) was built inside a `bwrap` (bubblewrap) sandbox on the project's machine: the system directories bound read-only, a private `tmpfs` home with nothing of the real home or of this repository visible, separate PID, IPC and UTS namespaces, the whole sandbox confined to eight cores (`taskset -c 0-7`) at the lowest scheduling priority, and the network on, because the repository's `lakefile` clones its 41 dependencies at build time and the Mathlib object files come from the Mathlib cache. Inside, `elan` was installed into the private home and fetched the pinned toolchain from the repository's own `lean-toolchain` file; then, in the clone's `lean/` directory, `lake exe cache get`, `lake build OAI.Algorithms.SubexponentialSAT.Main` (the solution's closure, not the collection's default target, which builds all 721 families), and `#print axioms` of the one theorem the comparator names, `OAI.Algorithms.SubexponentialSAT.SqrtLogSpace.exact_count_and_sat`, through `lake env lean`. Only text left the sandbox; nothing built inside it was executed outside it.

Two runs were needed. The first, started at 01:03 on 2026-11-05, failed at once at the build step because its command line carried `-j 8`, an option this version of Lake does not have (the whole of its build log is the one line `error: unknown short option '-j'`); nothing of the family was compiled. The second, started at 01:16 with that option removed and the core cap moved to `taskset`, is the run reported below. Both scripts, both versions, and the launch command are in the README; the first run's logs are kept beside the second's.

5.2 What came out

The build finished with exit code 0 in 33 minutes of wall clock on eight cores (01:16:25 to 01:49:31), compiling **3,091** modules with the prefix `OAI.Algorithms.SubexponentialSAT`—exactly the import closure the inspection of Section 4 enumerated, the same 3,091 names with an empty set difference both ways (`paper3/replay/counts.sh`; Lake's own job total, `12014`, counts the cached Mathlib and dependency modules, none of which was compiled). The log has no line containing `error` and none containing `sorry`; its eleven `warning` lines are Lake noting that the lakefile's compatibility patches leave eleven dependencies with "local changes", and none of the eleven is Mathlib. Seventeen source files of the closure themselves carry `#print axioms` commands, so the build log records nineteen axiom lines as it goes, every one `[propext, Classical.choice, Quot.sound]`; among them `SqrtLogSpace/Final.lean`, lines 197–201, prints the axioms of `compiled_exact_count_and_sat`, `exact_count_and_sat`, `exact_count_eth`, `sat_polynomial_space` and `original_sat_main`. The separate `#print axioms` run after the build gives, for the comparator's theorem,

```
'OAI.Algorithms.SubexponentialSAT.SqrtLogSpace.exact_count_and_sat'
depends on axioms:  [propext, Classical.choice, Quot.sound]
```

(`paper3/replay/logs/run2/axioms.txt`). The versions are `Lean (version 4.34.1, commit 5045d005..., Release)`, `Lake version 5.0.0-src+5045d00`, and Mathlib at `d13f23b723b8a846827` from the clone's own manifest, with its object files from the Mathlib cache (8,908 files). The replay copy is at commit `4eaee875`, the same as the pristine clone, and the comparator file and the solution's files hash identically in both (`paper3/replay/logs/run2/hashes.txt`; the comparator statement's SHA-256 begins `4183b205`). The full build log is kept out of the repository by size; its SHA-256 (`d313f594...`) and its last 200 lines are committed.

So the sentence every later section relies on reads: *replayed*—the solution's closure builds from source at the pinned toolchain with upstream Mathlib, and the kernel reports the three standard axioms and nothing else for the theorem the comparator names. This moves the first three items of the inspection's trust boundary (Section 4): the build is no longer OpenAI's unpublished one, the `#print axioms` output is now on record, and the Mathlib revision was fetched from upstream and not patched.

5.3 The second kernel

The comparator's JSON for the family has `enable_nanoda: false`: OpenAI's check of this family ran the Lean kernel only. We ran a second kernel, in the same sandbox, after the build (`paper3/replay/nanoda.sh`, recorded in the README before it ran): `lean4export` (tag `v4.34.0`, commit `076e8e57`, built with the pinned `v4.34.1` toolchain so that it reads the object files the build produced) exported `exact_count_and_sat` and its transitive dependencies from the built environment—2,355,666,449 bytes of NDJSON, 43,633,742 lines, format version 3.1.0, Lean githash `5045d005...`, in 68 seconds—and `nanoda_lib` (nanoda contributors, 2026) (version 0.4.31, commit `8de7e0c4`, 2026-10-29; an external type checker for Lean 4 written in Rust, which shares no code with the Lean kernel) checked that export with `propext`, `Classical.choice` and `Quot.sound` as the permitted axioms and the `Nat` and `String` literal extensions on. Its output (`paper3/replay/logs/nanoda/nanoda_check.txt`):

```
Checked 95176 declarations with no errors
```

in 110 seconds on eight cores, exit code 0, after printing back the theorem's statement and the three axioms from its own parse of the export. The export contains exactly three `axiom` declarations (`grep -c '"axiom"'` on the file), so nothing else could have been permitted. The export file (SHA-256 `bdbc9afe...`, `paper3/replay/logs/nanoda/export_sha256.txt`) stays in the sandbox directory by size; the configuration, the build logs and the checker's output are committed.

5.4 What this does not claim

- The replay trusts the Lean kernel of the pinned toolchain (`v4.34.1`) and upstream Mathlib at the pinned revision; it moves the proof's trust from "OpenAI's unpublished build" to "a build we ran", not from the Lean kernel to anything else. The second kernel removes the Lean kernel from the list for the theorem's dependencies as exported, but adds `lean4export` (that it exports what the environment holds) and `nanoda_lib` (that it checks what it parses) to it; two independent checkers agreeing is evidence, not a proof that either is correct.
- It is not the comparator run. The comparator checks that the solution's exported statement matches the challenge's by `lean4export`; we did not run that tool chain, and the statement-level match the paper relies on is the textual one of Section 4 (the 33 definitions and the theorem statement, identical character for character after whitespace normalisation).
- The counts above are counts of lines in one build's log on one machine with the named versions; a different Lake would print different job totals (the 12,014 includes the cached Mathlib modules), and a different machine a different time.
- Nothing here bears on what the theorem *says* (Section 2) or on the manuscript's unverified exponents (Section 3); a replayed proof of a weak bound is a replayed proof of a weak bound.

6 What follows

Placeholder. Item 05 drafts this section: which hypotheses the verified statement refutes (ETH, SETH, Gap-ETH, #ETH, NSETH, the randomised forms) and how; in general terms, that a lower bound derived from ETH or SETH loses its premise while a statement "no $(c-\varepsilon)^k$ algorithm" is not itself refuted (the manuscript's §1.1.3); no example from any width parameter. Ends with what this does not claim".

7 For the practitioner

Placeholder. Item 06 drafts this section: a SAT solver and a model counter on random 3-CNF at $n = 20 \ldots 60$ against $2^{\rho(n)}$ and 2^n (figure labelled `fig:practitioner`, from `paper3/scripts/practitioner.py`); the verified guarantee is never the binding one on any machine. Ends with what this does not claim".

8 The statement in our Lean

Placeholder. Item 07 drafts this section: the restatement `SubexpCount` over the verbatim machine model under `lean/MOSPFormalization/ETH/`, the axiom audit, the byte-level rediff against the comparator file; no theorem of ours about the hypothesis is proved, and none is claimed. Ends with what this does not claim".

9 Related work and context

Placeholder. Item 08 drafts this section: the human audit of the August 2026 results (arXiv 2608.14673), the working group named in the collection's history, Impagliazzo–Paturi, Impagliazzo–Paturi–Zane, the comparator mechanism, nanoda; "we found no prior statement", never "first". Ends with what this does not claim".

10 Conclusion

Placeholder. Item 09 drafts this section: what is claimed and what is not; every entry of `paper3/ETH\_AMBIGUOUS.MD` that affects the text, listed as open by number. Ends with what this does not claim".

Data, code and proofs

Placeholder. Item 11 completes this statement: the repository, the scripts under `paper3/scripts/` that regenerate every number, the replay's commands and log hashes under `paper3/replay/`, the Lean names and the axiom audit under `paper3/lean/`, and the read-only clone of `https://github.com/openai/math` at commit `4eaee875`.

Acknowledgements

Placeholder. Item 11 writes the acknowledgements, with the disclosure of AI use in the form of `paper1/latex/sec8\_ack.tex`.

References

nanoda contributors. nanoda: a type checker for Lean 4 written in Rust. https://github.com/ammkrn/nanoda_lib, 2026. Repository README; author, version and date recorded by item 08.

Appendix A Formal names

Every Lean name the paper cites, with its file under `lean/MOSPFormalization/ETH/`; the axiom audit is `paper3/lean/axiom_check.lean` with its committed log.

	result	Lean names
A.1	The comparator statement restated as a `Prop`, `SubexpCount` k, over the verbatim copy of the machine model; never proved, never an axiom	`MOSPFormalization.SubexpPathwidth.SubexpCount`
A.2	$\rho(n) = o(n)$: the verified exponent is subexponential in the limit	`MOSPFormalization.SubexpPathwidth.runtimeScale_isLittleO`

Placeholder. Item 07 moves these declarations to `lean/MOSPFormalization/ETH/` and rewrites the names above to their new namespace.